

# Stajerlerin Ekrana Yazdırılması

## Teknik Dokümantasyon

### Stajer Manager Projesi

## İçindekiler

- Genel Bakış
- Routing ve Component Yapısı
- Backend API Katmanı
- Frontend Service Katmanı
- State Yönetimi ve Filtreleme
- Render İşlemi
- Veri Akış Diyagramı
- Önemli Teknik Detaylar

## 1. Genel Bakış

### Mimari Yapı

- Frontend:** React.js (SPA)
- Backend:** ASP.NET Core Web API
- Veritabanı:** Entity Framework Core
- HTTP İletişimi:** Axios

### Temel Akış

Kullanıcı → React Component → Service → API → Controller → Database

## 2. Routing ve Component Yapısı

### Route Tanımı (App.jsx)

```
import { Routes, Route } from 'react-router-dom'
import StajersPage from './pages/StajersPage'

function App() {
  return (
    <Routes>
      <Route
        path="/stajers"
        element={
          <PrivateRoute>
            <StajersPage />
          </PrivateRoute>
        }
      />
    </Routes>
  )
}
```

### PrivateRoute Component

```
function PrivateRoute({ children }) {  
  const { user, loading } = useAuth()  
  
  if (loading) {  
    return <div>Yükleniyor...</div>  
  }  
  
  return user ? children : <Navigate to="/login" replace />  
}
```

## Component Hiyerarşisi

- App.jsx → Routing yönetimi
- StajersPage.jsx → Ana sayfa component'i
- stajersService.js → API çağrıları
- StajersApiController.cs → Backend endpoint

---

## 3. Backend API Katmanı

### Controller Endpoint (StajersApiController.cs)

```

[ApiController]
[Route("api/[controller]")]
public class StajersApiController : ControllerBase
{
    private readonly Context _context;
    private readonly ILogger<StajersApiController> _logger;

    [HttpGet]
    [AllowAnonymous]
    public async Task<IActionResult> GetStajers(
        [FromQuery] string? sortBy,
        [FromQuery] string? sortOrder,
        [FromQuery] string? searchText)
    {
        sortBy ??= "StajerID";
        sortOrder = (sortOrder?.ToLower() == "asc") ? "asc" : "desc";

        var query = _context.Stajers
            .Include(s => s.Departman)
            .Include(s => s.Universite)
            .Include(s => s.Bolum)
            .AsQueryable();

        // Arama filtresi
        if (!string.IsNullOrEmpty(searchText))
        {
            var lower = searchText.Trim().ToLower();
            query = query.Where(s =>
                s.FullName.ToLower().Contains(lower) ||
                s.Email.ToLower().Contains(lower) ||
                s.PhoneNumber.Contains(lower) ||
                (s.Universite != null && s.Universite.UniversiteAdi.ToLower().Contains(lower)) ||
                (s.Bolum != null && s.Bolum.BolumAdi.ToLower().Contains(lower)) ||
                (s.Departman != null && s.Departman.DepartmanAdi.ToLower().Contains(lower))
            );
        }

        // Sıralama
        query = (sortBy, sortOrder) switch
        {
            ("FullName", "asc") => query.OrderBy(s => s.FullName),
            ("FullName", "desc") => query.OrderByDescending(s => s.FullName),
            ("StajerID", "asc") => query.OrderBy(s => s.StajerID),
            _ => query.OrderByDescending(s => s.StajerID)
        };

        var stajers = await query.ToListAsync();
        var response = stajers.Select(MapStajerListItem).ToList();

        return Ok(new
        {
            success = true,
            stajers = response,
            totalCount = response.Count
        });
    }
}

```

## İşlem Adımları

1. **Include ilişkileri:** Departman, Üniversite, Bolum
2. **Filtreleme:** SearchText varsa WHERE koşulu
3. **Sıralama:** SortBy ve SortOrder'a göre
4. **Mapping:** Entity → DTO dönüşümü
5. **Response:** JSON formatında dönüş

---

## 4. Frontend Service Katmanı

---

### API Service (stajerService.js)

```
import { api } from '../lib/api.js'

export const stajerService = {
  // Tüm stajerleri getir (arama ve sıralama ile)
  getAll: async (sortBy = 'StajerID', sortOrder = 'desc', searchText = '') => {
    const response = await api.get('/StajersApi', {
      params: {
        sortBy,
        sortOrder,
        searchText: searchText || ''
      }
    })
    return response.data
  },

  // Tek bir stajer getir
  getById: async (id) => {
    const response = await api.get(`/StajersApi/${id}`)
    return response.data?.data || response.data
  },

  // Yeni stajer oluştur
  create: async (stajer) => {
    const response = await api.post('/StajersApi', stajer)
    return response.data
  }
}

export default stajerService
```

### Axios Konfigürasyonu (lib/api.js)

```
import axios from 'axios'

const api = axios.create({
  baseURL: 'http://localhost:5203/api',
  withCredentials: true,
  headers: {
    'Content-Type': 'application/json'
  }
})

export default api
export { api }
```

### Özellikler

- **Base URL:** http://localhost:5203/api
- **Credentials:** withCredentials: true (Cookie desteği)
- **Content-Type:** application/json
- **Interceptors:** Request/Response interceptors eklenebilir

---

## 5. State Yönetimi

---

### State Tanımlamaları (StajersPage.jsx)

```
import { useState, useEffect, useCallback } from 'react'
import stajService from '../services/stajService.js'
import { isStajActive } from '../utils/stajUtils.js'

function StajersPage() {
  // Veri state'leri
  const [stajers, setStajers] = useState([]) // Filtrelenmiş stajyerler
  const [allStajers, setAllStajers] = useState([]) // Tüm stajyerler (backend'den gelen)

  // UI state'leri
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [searchText, setSearchText] = useState('')
  const [sortBy, setSortBy] = useState('StajyerID')
  const [sortOrder, setSortOrder] = useState('desc')
  const [totalCount, setTotalCount] = useState(0)
  const [IsActive, setIsActive] = useState(false)

  // Modal state'leri
  const [showCreateModal, setShowCreateModal] = useState(false)
  const [showEditModal, setShowEditModal] = useState(false)
  const [selectedStajyer, setSelectedStajyer] = useState(null)

  // Auth context
  const { user, loading: authLoading } = useAuth()
  const isAdmin = !authLoading && (user?.role === 'admin')

  // ... component logic ...
}
```

## İki Aşamalı Filtreleme

1. **Backend:** Sıralama ve opsiyonel arama
2. **Frontend:** Aktif stajyer filtresi + arama

## 6. Filtreleme Mekanizması

### Aktif Stajyer Kontrolü (utils/stajUtils.js)

```
export const isStajActive = (stajyer) => {
  if (!stajyer?.startDate || !stajyer?.endDate) {
    return false
  }

  const today = new Date()
  today.setHours(0, 0, 0, 0)

  const startDate = new Date(stajyer.startDate)
  startDate.setHours(0, 0, 0, 0)

  const endDate = new Date(stajyer.endDate)
  endDate.setHours(0, 0, 0, 0)

  return today >= startDate && today <= endDate
}
```

### Filtreleme Fonksiyonu (StajersPage.jsx)

```
// Stajyerleri filtrele
const filterStajers = useCallback((stajersList) => {
  let filtered = [...stajersList]

  // Aktif stajyer filtresi
  if (IsActive) {
    filtered = filtered.filter(isStajActive)
  }

  // Arama filtresi
  if (searchText.trim()) {
    const lower = searchText.trim().toLowerCase()
    filtered = filtered.filter(s =>
      s.fullName?.toLowerCase().includes(lower) ||
      s.email?.toLowerCase().includes(lower) ||
      s.phoneNumber?.includes(lower) ||
      s.universite?.universiteAdi?.toLowerCase().includes(lower) ||
      s.bolum?.bolumAdi?.toLowerCase().includes(lower) ||
      s.departman?.departmanAdi?.toLowerCase().includes(lower)
    )
  }

  return filtered
}, [IsActive, searchText])
```

## Arama Filtresi Kapsamı

- Ad Soyad ( fullName )
- Email ( email )
- Telefon ( phoneNumber )
- Üniversite ( universite.universiteAdi )
- Bölüm ( bolum.bolumAdi )
- Departman ( departman.departmanAdi )

---

## 7. Render İşlemi

### Veri Yükleme Fonksiyonu

```
const loadStajers = async () => {
  try {
    setLoading(true)
    setError(null)
    const data = await stajerService.getAll(
      sortBy,
      sortOrder,
      '' // Arama metnini backend'e göndermiyor
    )

    if (data && data.stajers) {
      setAllStajers(data.stajers) // Tüm stajerleri kaydet
      const filtered = filterStajers(data.stajers) // Filtrele
      setStajers(filtered)
      setTotalCount(filtered.length)
    } else if (Array.isArray(data)) {
      setAllStajers(data)
      const filtered = filterStajers(data)
      setStajers(filtered)
      setTotalCount(filtered.length)
    } else {
      setAllStajers([])
      setStajers([])
      setTotalCount(0)
    }
  } catch (err) {
    console.error('Stajer yükleme hatası:', err)
    setError('Stajerler yüklenirken hata oluştu: ' + (err.response?.data?.message || err.message))
    setAllStajers([])
    setStajers([])
  } finally {
    setLoading(false)
  }
}
```

## Tablo Yapısı (JSX)

```

<tbody>
  {stajers.length === 0 ? (
    <tr>
      <td colSpan="10" className="text-center py-4 text-muted">
        {searchText ? 'Arama sonucu bulunamadı.' : 'Henüz stajer eklenmemiş.'}
      </td>
    </tr>
  ) : (
    stajers.map((s) => (
      <tr key={s.stajerID} className={!isStajActive(s) ? 'table-danger' : ''}>
        <td><span className="badge bg-secondary">{s.stajerID}</span></td>
        <td>{s.fullName}</td>
        <td>{s.email}</td>
        <td>{s.phoneNumber}</td>
        <td>{s.universite?.universiteAdi || s.universite || '-'}</td>
        <td>{s.bolum?.bolumAdi || s.bolum || '-'}</td>
        <td>{s.departman?.departmanAdi || s.departman || '-'}</td>
        <td>{formatDate(s.startDate)}</td>
        <td>{formatDate(s.endDate)}</td>
        <td className="text-center">
          <div className="d-flex justify-content-center gap-2">
            <button
              className="btn btn-info btn-sm"
              onClick={() => openDetailsModal(s)}
              title="Detaylar"
            >
              <i className="fas fa-eye"></i>
            </button>
            {isAdmin && (
              <>
                <button
                  className="btn btn-warning btn-sm"
                  onClick={() => openEditModal(s)}
                  title="Düzenle"
                >
                  <i className="fas fa-edit"></i>
                </button>
                <button
                  className="btn btn-danger btn-sm"
                  onClick={() => openDeleteModal(s)}
                  title="Sil"
                >
                  <i className="fas fa-trash"></i>
                </button>
              </>
            )}
          </div>
        </td>
      </tr>
    ))
  )}
</tbody>

```

## Tarih Formatlama



```
const formatDate = (dateString) => {
  if (!dateString) return '-'
  try {
    const date = new Date(dateString)
    if (isNaN(date.getTime())) return dateString
    return date.toLocaleDateString('tr-TR')
  } catch {
    return dateString
  }
}
```

## Özellikler

- **Key Prop:** React için unique identifier ( `s.stajerID` )
- **Conditional Styling:** Aktif olmayanlar kırmızı ( `table-danger` )
- **Formatting:** Tarihler Türkçe formatında ( `tr-TR` )
- **Action Buttons:** Detay, Düzenle, Sil (Admin only)
- **Null Safety:** Optional chaining ( `?.` ) ve fallback değerler

## 8. Veri Akış Diyagramı



## 9. Önemli Teknik Detaylar

### Performance Optimizasyonları

```
// useCallback ile memoization
const filterStajers = useCallback((stajersList) => {
  // ... filtreleme logic ...
}, [IsActive, searchText])

// Conditional rendering
{!loading && (
  <div className="table-responsive">
    {/* Tablo içeriği */}
  </div>
)}

// Dependency array kontrolü
useEffect(() => {
  loadStajers()
}, [sortBy, sortOrder]) // Sadece gerekli değişkenler
```

## Error Handling

```
// Try-catch bloğu
try {
  setLoading(true)
  const data = await stajerService.getAll(sortBy, sortOrder, '')
  // ... veri işleme ...
} catch (err) {
  console.error('Stajer yükleme hatası:', err)
  setError('Stajerler yüklenirken hata oluştu: ' +
    (err.response?.data?.message || err.message))
} finally {
  setLoading(false)
}
```

## Security

```
// PrivateRoute ile authentication
<PrivateRoute>
  <StajersPage />
</PrivateRoute>

// Role-based access control
const isAdmin = !authLoading && (user?.role === 'admin')

{isAdmin && (
  <button onClick={openCreateModal}>
    Yeni Stajer Ekle
  </button>
)}
```

## Best Practices

- **Memoization:** useCallback ile gereksiz render'ları önleme
- **Error Boundaries:** Hata yakalama ve kullanıcıya gösterme
- **Loading States:** Kullanıcı deneyimi için yükleme göstergeleri
- **Input Validation:** Backend ve frontend'de doğrulama
- **Type Safety:** Optional chaining ve null checks

# 10. Mapping Fonksiyonu

## Backend Mapping (StajersApiController.cs)

```

private static object MapStajerListItem(StajerModel s) => new
{
    stajerID = s.StajerID,
    fullName = s.FullName,
    email = s.Email,
    phoneNumber = s.PhoneNumber,
    universiteID = s.UniversiteID,
    bolumID = s.BolumID,
    departmanID = s.DepartmanID,
    universite = s.Universite != null
        ? new { universiteAdi = s.Universite.UniversiteAdi }
        : null,
    bolum = s.Bolum != null
        ? new { bolumAdi = s.Bolum.BolumAdi }
        : null,
    departman = s.Departman != null
        ? new { departmanAdi = s.Departman.DepartmanAdi }
        : null,
    startDate = s.StartDate,
    endDate = s.EndDate,
    notes = s.Notes
};

private static object MapStajerDetail(StajerModel s) => new
{
    stajerID = s.StajerID,
    fullName = s.FullName,
    email = s.Email,
    phoneNumber = s.PhoneNumber,
    universiteID = s.UniversiteID,
    bolumID = s.BolumID,
    departmanID = s.DepartmanID,
    startDate = s.StartDate,
    endDate = s.EndDate,
    notes = s.Notes,
    universite = s.Universite != null
        ? new { s.Universite.UniversiteAdi }
        : null,
    bolum = s.Bolum != null
        ? new { s.Bolum.BolumAdi }
        : null,
    departman = s.Departman != null
        ? new { s.Departman.DepartmanAdi }
        : null
};

```

## Frontend Null Safety

```

// Optional chaining
{s.universite?.universiteAdi}

// Null coalescing
{user?.role ?? 'user'}

// Fallback değerler
{s.universite?.universiteAdi || s.universite || '-'}

```

## Null Safety Teknikleri

- **Optional Chaining (?.):** Null/undefined kontrolü
- **Null Coalescing (??):** Varsayılan değer atama
- **Fallback Değerler (|| '- '):** Alternatif gösterim
- **Ternary Operator:** Koşullu render

## 11. Kullanıcı Deneyimi

### Loading States

```
{loading && (  
  <div className="text-center my-4">  
    <div className="spinner-border text-primary"></div>  
    <p className="mt-2">Veriler yükleniyor...</p>  
  </div>  
)}
```

### Error States

```
{error && (  
  <div className="alert alert-danger d-flex justify-content-between">  
    <span>{error}</span>  
    <button  
      className="btn btn-outline-light btn-sm"  
      onClick={loadStajers}>  
    >  
      Veriler Yüklenemedi. Tekrar Dene  
    </button>  
  </div>  
)}
```

### Empty States

```
{stajers.length === 0 && !loading && (  
  <tr>  
    <td colSpan="10" className="text-center py-4 text-muted">  
      {searchText  
        ? 'Arama sonucu bulunamadı.'  
        : 'Henüz stajer eklenmemiş.'}  
    </td>  
  </tr>  
)}
```

### useEffect Hook'ları

```
// İlk yükleme  
useEffect(() => {  
  loadStajers()  
}, [sortBy, sortOrder])  
  
// Otomatik filtreleme  
useEffect(() => {  
  if (allStajers.length > 0) {  
    const filtered = filterStajers(allStajers)  
    setStajers(filtered)  
    setTotalCount(filtered.length)  
  }  
}, [isActive, searchText, allStajers, filterStajers])
```

### UX Özellikleri

- **Loading Spinner:** Veri yüklenirken gösterge
- **Error Messages:** Anlaşılır hata mesajları
- **Retry Button:** Hata durumunda yeniden deneme
- **Empty States:** Boş durum mesajları
- **Search Feedback:** Arama sonuç sayısı gösterimi

---

## 12. Sonuç

---

### Özet

- 🔖 **Mimari:** Modern React + ASP.NET Core
- 🔖 **Performans:** Optimize edilmiş filtreleme
- 🔖 **UX:** Loading, error, empty states
- 🔖 **Security:** Authentication & Authorization
- 🔖 **Maintainability:** Clean code principles

### Teknoloji Stack

- React.js (Hooks, Context API)
- ASP.NET Core (Web API)
- Entity Framework Core
- Axios
- Bootstrap 5

---

## Sorular?

---

Teşekkürler!